

CO5 – ASSESSMENT 1

CONSTRAINT-BASED PROBLEM SOLVING

NAME: A.ARCHISHMAN

REGNO: 192425341

COURSE CODE: ITA0502

COURSE NAME: COMPUTER VISION

Q1. Real-Time Face Recognition for Campus Security

Objective:

Design a complete OpenCV-based pipeline that captures live CCTV video, detects multiple faces under different lighting conditions, extracts suitable facial features, recognizes registered students/staff and maintains real-time performance on a low-cost computer while minimizing false recognition.

1. Image Acquisition:

Use OpenCV VideoCapture to continuously read frames from the CCTV/IP camera. Resize frames to a practical resolution and define an entrance Region of Interest (ROI) so that unnecessary parts of the image are not processed.

2. Preprocessing:

Convert the frame or face crop to grayscale when using a classical detector. Apply mild Gaussian filtering to reduce sensor noise and CLAHE/histogram equalization to improve contrast under uneven lighting. Align the face using eye positions when landmarks are available.

3. Face Detection:

Use a Haar Cascade for a lightweight CPU-based solution or an OpenCV DNN face detector for improved robustness. Detect all faces in the frame and reject very small or low-quality detections.

4. Feature Extraction:

For a classical OpenCV solution, use LBP or HOG features. A stronger option is an OpenCV-compatible deep face embedding model. Store several feature templates for every registered person so that different poses and lighting conditions are represented.

5. Recognition:

Compare the live feature vector with registered templates using Euclidean or cosine distance. Select the closest reliable match. If the similarity does not satisfy a validated threshold, classify the person as Unknown instead of forcing a wrong identity.

6. False Recognition Control:

Use multiple enrollment images, face-quality checks, conservative recognition thresholds and confirmation across consecutive frames. This prevents a single incorrect frame from immediately producing a false identity.

7. Real-Time Optimization:

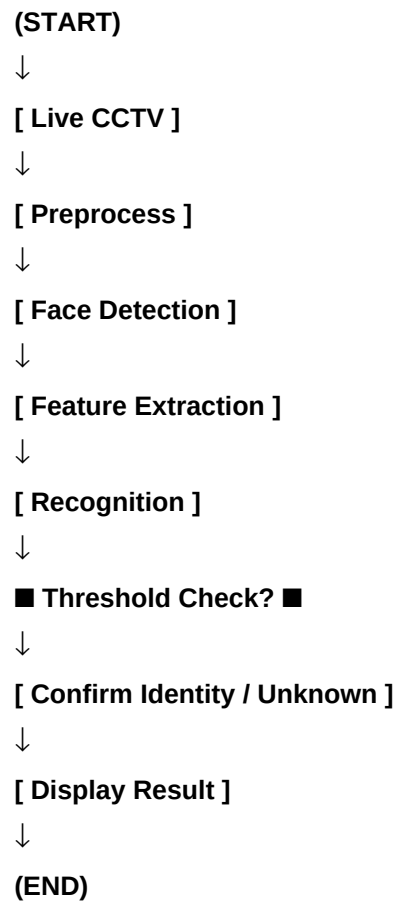
Process a smaller frame, detect faces every few frames and track faces between detections. Process only the ROI and face crops, cache recent identities and use OpenCV optimized CPU/DNN operations where available.

Conclusion:

The combination of preprocessing, robust face detection, feature extraction, threshold-based recognition and temporal verification improves accuracy. Frame resizing, ROI processing and detection-plus-tracking reduce

computation and support real-time operation on low-cost hardware.

Flowchart:



Q2. Intelligent Traffic Violation Detection System

Objective:

Design an OpenCV system that detects vehicles in live video, tracks them across frames, counts line crossings, identifies abnormal movements and displays real-time counts and alerts.

1. Video Acquisition:

Capture live roadside video with OpenCV VideoCapture. Define a road ROI so that vehicles are processed while irrelevant areas such as buildings and trees are excluded.

2. Preprocessing:

Resize frames and apply mild Gaussian or median filtering to reduce camera noise. For a fixed camera, convert the image to a suitable representation for motion analysis.

3. Vehicle Detection:

Use an OpenCV DNN object detector to identify cars, buses, trucks and motorcycles. In fixed-camera scenes, MOG2/KNN background subtraction can also provide foreground candidates.

4. Tracking:

Assign a unique ID to every detected vehicle and maintain its bounding box/centroid across consecutive frames using a multi-object tracker such as centroid tracking or a Kalman-filter-based tracker.

5. Line-Crossing and Counting:

Draw a virtual line across the road. Compare the previous and current centroid positions of each track. When a vehicle crosses the line in the required direction, increase the count only if its track ID is not already in the counted set. This prevents double counting.

6. Abnormal Movement Detection:

Analyze the vehicle trajectory, direction and approximate speed to detect wrong-way movement, illegal turns, sudden stops or movement outside a permitted region. Generate an alert when a defined traffic rule is violated.

7. Handling Shadows and Dynamic Backgrounds:

Use ROI restriction, thresholding, morphological opening/closing and temporal smoothing to remove small foreground noise. Shadow suppression reduces false detections. Since the camera is roadside and the ROI is limited to the road, trees and unrelated background motion have less influence.

8. Visualization:

Display each vehicle's bounding box and tracking ID, the virtual counting line, total vehicle count and violation alerts. Store important events with timestamps for later review.

Conclusion:

Detection, tracking and trajectory analysis provide reliable counting and violation detection. Unique track IDs prevent duplicate counts, while ROI processing, filtering and shadow suppression reduce errors and maintain near real-time performance.

Flowchart:



↓

[Vehicle Detection]

↓

[Tracking + IDs]

↓

[Line Crossing Check]

↓

[Count Once]

↓

[Violation Analysis]

↓

[Display Count + Alert]

↓

(END)

Q3. Automated Document Processing System

Objective:

Create an OpenCV pipeline that improves scanned document quality, removes noise/background, prepares images for OCR, segments text regions and supports efficient batch processing.

1. Image Acquisition and Batch Processing:

Read documents from a folder and process them one at a time. Save the final OCR-ready images or extracted results using unique filenames.

2. Grayscale Conversion:

Convert each document from RGB/BGR to grayscale. This reduces the image to one intensity channel and simplifies filtering, thresholding and segmentation.

3. Noise Removal:

Use median filtering for salt-and-pepper noise and Gaussian filtering for mild random noise. The filter strength should be limited so that character edges and thin strokes are preserved.

4. Illumination Correction and Thresholding:

Use adaptive thresholding when illumination is uneven. Otsu thresholding is effective when the page background is relatively uniform. Thresholding separates text from the background and creates a cleaner OCR input.

5. Morphological Operations:

Apply opening to remove isolated small noise and closing to fill small gaps or reconnect broken text structures. Excessive morphology must be avoided because it can remove character strokes.

6. Text Segmentation:

Use contours, connected components or projection profiles to locate text blocks and lines. Non-text regions can be removed or ignored before OCR, reducing unnecessary processing.

7. Geometric Correction:

Estimate document orientation and apply deskewing when text is rotated. If the document is photographed at an angle, detect the page corners and apply a perspective transformation to obtain a rectangular page.

8. OCR Preparation:

Resize the corrected image if necessary and maintain a clean text/background contrast. The final image is then passed to an OCR engine such as Tesseract or another OCR system.

9. Batch Optimization:

Reuse preprocessing settings for documents of the same type, avoid unnecessary conversions and use parallel processing where resources allow. This increases throughput for thousands of documents.

Conclusion:

Grayscale simplifies processing, filtering removes noise, thresholding separates text, morphology repairs structures, segmentation isolates text and geometric correction straightens the page. Together they improve OCR accuracy and batch-processing efficiency.

Q4. Real-Time Multi-Person Tracking and Attendance System

Objective:

Build a classroom CCTV system that detects multiple faces, tracks people across frames, recognizes registered students, records attendance only once and continues operating with low delay.

1. Face Detection:

Capture frames with OpenCV and restrict processing to the classroom ROI. Detect all visible faces using a Haar Cascade or OpenCV DNN detector.

2. Preprocessing and Feature Extraction:

Normalize illumination, reject very small or blurred faces and align faces when landmarks are available. Extract LBP/HOG features or deep face embeddings and compare them with registered student templates.

3. Recognition:

Use a conservative similarity/distance threshold. Confirm a student's identity only when the recognition result is reliable; otherwise keep the track as Unknown.

4. Identity Tracking:

Assign a unique track ID to each detected person. Use centroid tracking, optical flow or a Kalman-filter-based tracker to maintain the identity between detection frames.

5. Temporary Face Disappearance:

Keep a track alive for a short timeout when the face temporarily disappears because of slight head movement or brief occlusion. When a matching face reappears near the predicted location, reconnect it to the existing track.

6. Preventing Duplicate Attendance:

Maintain an attendance set/dictionary keyed by student ID. After multi-frame identity confirmation, record the student ID and timestamp only if that ID has not already been marked Present.

7. Real-Time Optimization:

Detect faces every few frames and track between detections. Resize frames, process only the classroom ROI and cache recent recognition results for active tracks.

Conclusion:

Combining face recognition with tracking maintains identity across frames. Multi-frame confirmation reduces false recognition, while the attendance set prevents duplicate records. Detection skipping and ROI processing reduce the processing delay.

Q5. Smart Home Surveillance and Intrusion Detection

Objective:

Design an OpenCV-based fixed-camera surveillance pipeline that detects meaningful motion, tracks moving objects, identifies suspicious activity and generates an alert when an unauthorized person enters a protected region.

1. Video Acquisition and ROI:

Capture frames continuously using OpenCV VideoCapture. Define a protected ROI around the entrance so that irrelevant background regions are excluded.

2. Background Subtraction:

Use MOG2 or KNN background subtraction to estimate the static background and create a foreground mask for moving objects.

3. Filtering and Morphological Operations:

Apply Gaussian/median filtering to reduce sensor noise. Threshold the foreground mask and use morphological opening to remove small isolated blobs and closing/dilation to connect fragmented object regions.

4. Shadow and Illumination Handling:

Use shadow detection supported by the background subtractor when available. Combine motion with area and shape constraints and require movement to persist for several frames so that brief illumination changes do not create an alarm.

5. Contour and Feature Detection:

Find contours in the cleaned foreground mask and reject very small regions using an area threshold. For relevant objects, calculate the bounding box, centroid, size and movement direction.

6. Tracking and Suspicious Activity:

Assign track IDs and follow moving objects using centroid tracking or a Kalman-filter-based tracker. Suspicious activity can be defined as entering the protected polygon, remaining there for a specified time or moving during a restricted period.

7. Reducing False Alarms:

Trees and curtains can create dynamic background motion. ROI restriction, minimum-area thresholds, temporal persistence, motion direction and adaptive background modelling reduce these false alarms.

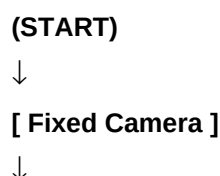
8. Real-Time Operation and Alerts:

Resize frames, process only the required ROI and track existing objects instead of performing expensive analysis from scratch on every frame. When an unauthorized entry is confirmed, display an alert and optionally save a timestamped image or short event video.

Conclusion:

Background subtraction combined with filtering, morphology, contours, tracking and rule-based analysis provides an efficient low-cost surveillance system. Temporal confirmation and background-noise suppression significantly reduce false alarms.

Flowchart:



[Background Subtraction]



[Filter + Morphology]



[Contours + Tracking]



[Protected ROI Check]



■ Intrusion Confirmed? ■



[Alert + Save Event / Continue]



(END)